GENERATIVE AI INTERNSHIP

MONTH-1 PROJECT REPORT

Domain: Generative AI

Tasks Completed

Task 1 :
Build a Streamlit Interface for Inference of a Locally Hosted LLM

Task 2:
Medical Fine-Tuning with Quantization and Low Rank Adaptation via Unsloth in Google Colab

Submitted By :

Name: GANDE SANNIHITHA

Internship Domain: Generative AI

Organization: Arch Technologies

Email: sannihitha.gande04@gmail.com

Phone: 9347157778

Internship Month

Month 1

Academic Year

2026

INDEX

S. No.	Contents	Page No.
1	**Acknowledgement**	3
2	**Project Overview**	4
3	**Introduction**	5-6
4	**Internship Objectives**	7
5	**Technologies and Tools Used**	8-13
6	**Task 1 – Streamlit Interface for Inference of a Locally Hosted LLM**	14-25
6.1	Task 1 Objective	
6.2	Problem Statement	
6.3	Task Requirements	
6.4	Technologies Used in Task 1	
6.5	System Architecture	
6.6	Environment Setup	
6.7	Ollama Setup and Local LLM	
6.8	Streamlit Application Development	
6.9	Streamlit–Ollama Communication	
6.10	Chat Interface and User Interaction	
6.11	Conversation History	
6.12	New Chat / Reset Functionality	
6.13	Additional Chat Management Features	
6.14	File and Image Upload Interface	
6.15	Code Implementation	
6.16	Working Demonstration	
6.17	Task 1 Results	
6.18	Task 1 Screenshots	
7	**Task 2 – Medical Fine-Tuning with Quantization and LoRA using Unsloth**	26-40
7.1	Task 2 Objective	
7.2	Problem Statement	
7.3	Task Requirements	
7.4	Introduction to LLM Fine-Tuning	
7.5	Quantization	
7.6	Low-Rank Adaptation (LoRA)	
7.7	QLoRA	
7.8	Unsloth	

S. No.	Contents	Page No.
7.9	Google Colab Environment	___
7.10	Base Model	___
7.11	Medical Dataset	___
7.12	Dataset Preparation	___
7.13	Dataset Formatting using Chat Template	___
7.14	4-Bit Quantization Configuration	___
7.15	LoRA Adapter Configuration	___
7.16	Training Configuration	___
7.17	Model Fine-Tuning	___
7.18	Training Monitoring	___
7.19	Saving the Fine-Tuned Adapter	___
7.20	Testing the Fine-Tuned Model	___
7.21	Sample Medical Queries and Responses	___
7.22	Task 2 Results	___
7.23	Task 2 Screenshots	___
8	**Integration of Task 1 and Task 2**	41-46___
8.1	Integration Overview	___
8.2	End-to-End Workflow	___
8.3	Model Inference Workflow	___
8.4	Final Application	___
9	**Code Implementation**	47-54___
9.1	Task 1 Code	___
9.2	Task 2 Code	___
9.3	Important Code Components	___
10	**Results and Observations**	55-56___
10.1	Task 1 Observations	___
10.2	Task 2 Observations	___
10.3	Overall Project Outcome	___
11	**Challenges Faced and Solutions**	___
11.1	Challenges in Task 1	___
11.2	Challenges in Task 2	___
11.3	Solutions Implemented	___
12	**Learning Outcomes**	___
13	**Future Enhancements**	___
14	**Conclusion**	___
15	**References**	___

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Arch Technologies** for providing me with the opportunity to participate in the **Generative AI Internship Program**.

This internship has provided me with valuable practical exposure to Generative Artificial Intelligence, Large Language Models, local model inference, Streamlit application development, model fine-tuning, quantization, Low-Rank Adaptation (LoRA), and QLoRA.

During the first month of the internship, I worked on two practical tasks. The first task involved developing a Streamlit interface for inference of a locally hosted Large Language Model using Ollama**.** The second task involved implementing medical language model fine-tuning using quantization and Low-Rank Adaptation through Unsloth in Google Colab**.**

I am thankful to Arch Technologies for providing the learning resources, project requirements, and opportunity to gain hands-on experience with these Generative AI technologies. The tasks helped me understand how Large Language Models can be integrated into applications and adapted for specialized domains.

Finally, I would like to express my appreciation to everyone who provided guidance and support throughout the internship and contributed to my learning experience.

PROJECT OVERVIEW

Generative AI Internship – Month 1

Generative Artificial Intelligence is a branch of Artificial Intelligence focused on generating new content such as text, images, audio, and code. Large Language Models are an important part of Generative AI and can be used to understand and generate human-like text responses.

As part of the **Generative AI Internship at Arch Technologies**, two practical tasks were assigned for Month 1. These tasks focused on two important areas of working with Large Language Models: **local** LLM inference and domain-specific model fine-tuning**.**

Task 1 focused on building an interactive Streamlit interface for a locally hosted Large Language Model running through Ollama. The required interface included a user input area, model-generated responses, communication between the Streamlit frontend and local LLM backend, conversation history, and reset functionality.

Task 2 focused on medical-domain fine-tuning using a QLoRA-based workflow with Unsloth in Google Colab. The task involved working with a medical question-answering dataset, 4-bit quantization, LoRA adapter configuration, training, saving the adapter, and testing the fine-tuned model with new medical queries.

Step 2.1 — Tasks Completed

Task 1 — (Local LLM Chatbot): Build a Streamlit Interface for Inference of a Locally Hosted LLM

The first task involved developing a web-based chatbot interface using Streamlit and connecting it to a locally hosted Large Language Model through Ollama. The application supports conversational interaction and chat-management functionality.

Task 2 — (Medical LLM Fine-Tuning): Medical Fine-Tuning with Quantization and Low Rank Adaptation via Unsloth in Google Colab

The second task involved implementing a QLoRA-based fine-tuning workflow for a medical language model. The workflow used a medical question-answering dataset, 4-bit quantization, LoRA-based parameter-efficient fine-tuning, training in Google Colab, and testing of the resulting model.

The completed work provided practical exposure to the development of an LLM-powered application as well as the process of adapting a pretrained language model for a specialized domain.

INTRODUCTION

3.1 Introduction to Generative AI

Generative Artificial Intelligence (Generative AI) is a branch of Artificial Intelligence that focuses on creating new content based on patterns learned from existing data. The generated content can include text, images, audio, code, and other forms of information.

Generative AI systems use advanced machine learning and deep learning models to understand patterns in their training data and generate new outputs based on user instructions or prompts. Large Language Models (LLMs) are one of the important technologies used in Generative AI for processing and generating natural language.

Generative AI has applications in areas such as content creation, conversational systems, coding assistance, design, automation, and information processing. During this internship, the focus was on understanding the practical implementation of Generative AI using Large Language Models.

This is aligned with the introductory description in the Arch Technologies assignment, which describes Generative AI as a branch of AI that can create content such as text, images, audio, and code.

3.2 Large Language Models

Large Language Models (LLMs) are machine learning models designed to process and generate human language. They can understand user prompts and generate responses based on the patterns and information learned during training.

LLMs can be used for applications such as chatbots, question answering, summarization, content generation, coding assistance, and domain-specific applications.

In this internship, an LLM was used as the backend of a conversational application in Task 1. The project also explored how a pretrained LLM can be adapted to a specialized medical domain through fine-tuning in Task 2.

3.3 Local LLM Inference

Local LLM inference refers to running a language model on a local computer rather than sending every user request to a remote cloud-based model.

In Task 1, a locally hosted LLM was accessed through **Ollama**, while **Streamlit** was used to create the user interface. This allowed users to interact with the model through a web-based chatbot interface.

The internship assignment required the Streamlit application to provide a text input area, display model-generated responses, communicate with the locally hosted LLM, maintain conversation history, and provide reset functionality.

3.4 LLM Fine-Tuning

Fine-tuning is a process in which a pretrained language model is further trained using a dataset from a specific domain or task. This allows the model to be adapted for specialized use cases.

In Task 2, the focus was on adapting a language model for a **medical question-answering domain**. The workflow used **4-bit quantization, Low-Rank Adaptation (LoRA), and QLoRA with Unsloth in Google Colab**.

The assigned workflow included loading a domain-specific medical dataset, configuring 4-bit quantized LoRA adaptation, performing training, saving the trained adapter, and testing the model with new medical queries.

3.5 Scope of the Month 1 Project

The Month 1 project covered two complementary aspects of working with Large Language Models:

1. LLM Application Development

A Streamlit-based chatbot application was developed to provide an interface for interacting with a locally hosted LLM through Ollama.

2. LLM Domain Adaptation

A QLoRA-based workflow was implemented to fine-tune a language model using medical question-answering data with Unsloth in Google Colab.

Together, these tasks provided practical exposure to both the **application/inference side** and the **model adaptation/fine-tuning side** of Generative AI.

OBJECTIVES

1. Understand Generative AI concepts and the role of Large Language Models in Generative AI applications.
2. Develop a Streamlit-based LLM application that provides an interactive interface for users.
3. Run and interact with a locally hosted LLM using Ollama.
4. Implement conversational functionality including user prompts, model responses, and conversation history.
5. Understand LLM fine-tuning and its purpose in adapting pretrained models to specialized domains.
6. Understand quantization techniques, particularly 4-bit quantization, for reducing model memory requirements.
7. Understand Low-Rank Adaptation (LoRA) as a parameter-efficient fine-tuning technique.
8. Implement a QLoRA-based medical fine-tuning workflow using Unsloth in Google Colab.
9. Prepare and format a medical question-answering dataset for model training.
10. Train and save a fine-tuned adapter and test the resulting model using new medical queries. 11. 11.Understand the complete LLM workflow, from local inference and application development to domain-specific fine-tuning.

TECHNOLOGIES AND TOOLS USED

The Month 1 Generative AI internship involved a combination of programming languages, development frameworks, local LLM tools, model fine-tuning libraries, datasets, and cloud-based computing resources. The technologies were used across the two assigned tasks to develop an LLM-based application and perform medical-domain fine-tuning.

5.1 Technologies Used

Technology / Tool	Purpose
Python	Main programming language used for application development, model handling, data processing, and fine-tuning.
Streamlit	Used to develop the interactive web-based chatbot interface for the locally hosted LLM.
Ollama	Used to run and interact with the Large Language Model locally.
Llama 3.2	Local Large Language Model used for inference in Task 1.
Google Colab	Cloud-based development and training environment used for Task 2.
NVIDIA Tesla T4 GPU	GPU resource used for efficient model fine-tuning in Google Colab.
Unsloth	Used to efficiently perform parameter-efficient fine-tuning of the language model.
Hugging Face	Used for accessing the pretrained model and medical dataset.
LoRA	Parameter-efficient fine-tuning technique used to train a small set of additional model parameters.
QLoRA	Fine-tuning approach combining 4-bit quantization with LoRA.
4-bit Quantization	Used to reduce the memory requirements of the pretrained model during fine-tuning.
Medical QA Dataset	Domain-specific dataset used for medical question-answering fine-tuning.
JSON	Used in Task 1 for storing persistent chat history.

5.2 Python

Python was the primary programming language used during the internship. It was used for developing the Streamlit application, communicating with the locally hosted LLM, processing the medical dataset, configuring the fine-tuning workflow, and testing the trained model.

Python provided access to libraries and frameworks required for both application development and machine learning.

5.3 Streamlit

Streamlit is a Python-based framework for creating interactive web applications.

In Task 1, Streamlit was used to develop the **Nexa Chatbot** interface. The application provides a user interface through which users can enter prompts and receive responses generated by the locally hosted LLM.

The Streamlit application also included conversational features such as:

- User prompt input
- Display of assistant responses
- Conversation history
- New chat functionality
- Chat management
- Persistent chat history
- File and image upload interface

The use of Streamlit allowed the LLM functionality to be presented through an accessible web-based interface rather than requiring users to interact directly with the command line.

5.4 Ollama

Ollama was used as the local runtime for running the Large Language Model in Task 1.

The application communicated with the locally running model through the Ollama interface. When the user entered a prompt, the Streamlit application sent the conversation messages to Ollama, which generated the model response.

The basic interaction followed this workflow:

User Prompt → Streamlit Application → Ollama → Local LLM → Generated Response → Streamlit Interface

The assignment specifically required the Streamlit application to communicate with a locally installed LLM through Ollama and provide an interactive interface with conversation history and reset functionality.

5.5 Llama 3.2

Llama 3.2 was used as the local language model for Task 1.

The model was accessed through Ollama using the model name: llama3.2

The Streamlit application sends the conversation messages to the model and receives the generated response. This enables the application to function as a conversational chatbot.

5.6 Google Colab

Google Colab was used as the development and training environment for Task 2.

The fine-tuning process requires significant computational resources. Google Colab provided access to a GPU environment, allowing the model to be loaded, configured, trained, and tested without requiring a dedicated local GPU.

The Task 2 implementation was performed using a **Tesla T4 GPU**.

5.7 Unsloth

Unsloth was used for the efficient fine-tuning workflow in Task 2.

It was used to load the pretrained model, configure parameter-efficient fine-tuning, apply LoRA adapters, and perform supervised fine-tuning.

The internship assignment specifically required the use of Unsloth for the medical fine-tuning workflow involving quantization and Low-Rank Adaptation.

5.8 LoRA

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique.

Instead of updating all parameters of the pretrained model, LoRA introduces trainable low-rank adapter parameters into selected parts of the model. This reduces the number of parameters that need to be trained.

In Task 2, LoRA was configured for multiple projection layers of the language model, including:

q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj

The LoRA configuration used:

r = 16
lora_alpha = 16
lora_dropout = 0

5.9 QLoRA

QLoRA combines **quantization** with **LoRA-based fine-tuning**.

In this project, the pretrained model was loaded using **4-bit quantization**, while LoRA adapters were used for parameter-efficient training.

The general workflow was:

Pretrained Model → 4-bit Quantization → LoRA Adapter → Fine-Tuning → Saved Adapter → Testing

The assignment required a QLoRA workflow involving 4-bit quantization, LoRA adapter configuration, training, saving the adapter, and testing it with new medical queries.

5.10 Hugging Face

Hugging Face was used as part of the model and dataset ecosystem for Task 2.

The pretrained model used in the project was:

```
unsloth/Llama-3.2-1B-Instruct
```

The medical question-answering dataset used was:

```
MedInstruct-52k
```

The dataset contains medical questions and corresponding answers that can be used for domain-specific model training.

5.11 Medical QA Dataset

The medical dataset used for Task 2 was **MedInstruct-52k**, obtained from the Hugging Face dataset repository. The dataset is designed for medical instruction-following and contains medical instructions paired with corresponding responses. It was used as the domain-specific dataset for adapting the pretrained Llama 3.2 1B Instruct model toward medical-domain tasks.

The dataset was downloaded using the following command:

```
!wget -q https://huggingface.co/datasets/casey-martin/MedInstruct/resolve/main/data/MedInstruct-52k.json
```

The downloaded JSON file was then loaded and prepared for supervised fine-tuning. The training data was formatted into a conversational structure suitable for the Llama chat template before being passed to the fine-tuning process.

Updated technology list

Where your document currently says:

```
prithvi1029/medquad-medical-qa
```

change it to:

`casey-martin/MedInstruct` – **MedInstruct-52k medical instruction dataset**

5.12 JSON for Chat History

JSON was used in Task 1 to maintain persistent chat history.

The application stored conversation information in:

```
chat_history.json
```

This allowed conversations to remain available beyond a single application interaction and supported features such as chat history management.

Task 1

User → Streamlit Interface→ Python Application→ Ollama → Llama 3.2→Generated Response→ Streamlit Chat Interface

Task 2

Medical QA Dataset
↓
Dataset Preparation
↓
Llama 3.2 1B Instruct
↓
4-bit Quantization

↓
LoRA Configuration
↓
QLoRA Fine-Tuning
↓
Unsloth + Google Colab + Tesla T4
↓
Saved Adapter
↓
Medical Query Testing

5.14 Summary

The technologies used during Month 1 provided practical exposure to different stages of working with Large Language Models. Streamlit and Ollama were used for building and running a local LLM application, while Google Colab, Unsloth, LoRA, QLoRA, and a medical question-answering dataset were used for domain-specific model fine-tuning.

Together, these tools supported the two main objectives of the internship: building an LLM-powered application and understanding the process of adapting an LLM to a specialized domain.

TASK 1 – STREAMLIT INTERFACE FOR INFERENCE OF A LOCALLY HOSTED LLM

6.1 Task 1 Objective

The objective of Task 1 was to build an interactive Streamlit interface for inference of a locally hosted Large Language Model (LLM)**.**

The application was required to provide a user-friendly interface through which users could enter text prompts and receive responses from a locally running LLM. The application also needed to communicate with the locally hosted model, maintain conversation history, and provide a reset or new-chat functionality.

In this project, **Streamlit** was used for developing the frontend interface, while **Ollama** was used to run the LLM locally. The local model used in the implementation was **Llama 3.2**.

The developed application was named **Nexa Chatbot**.

Main objectives of Task 1

1. Develop an interactive chatbot interface using Streamlit.
2. Run a Large Language Model locally using Ollama.
3. Connect the Streamlit application with the locally hosted LLM.
4. Allow users to enter prompts and receive generated responses.
5. Maintain conversation history during chatbot interactions.
6. Provide functionality for starting a new conversation or resetting the current chat.
7. Provide additional chat-management functionality to improve the user experience.

6.2 Problem Statement

Large Language Models are commonly accessed through online platforms and cloud-based APIs. However, applications can also be developed to interact with models running locally on a user's computer.The purpose of this task was to understand how a locally hosted LLM can be connected to a web-based user interface and used for conversational interaction.

The internship assignment required the development of a Streamlit application connected to a locally installed LLM through Ollama. The application needed to support text input, display generated responses, communicate with the model through a local API or endpoint, maintain conversation history, and provide a reset option.

6.3 Task Requirements

The requirements provided for Task 1 were used as the basis for developing the application.

The application was required to include:

- An interactive Streamlit interface.
- A locally installed Large Language Model.
- Ollama for local model interaction.
- A text input area for user prompts.
- A response area for displaying the generated output.
- Communication between the frontend and backend through a local API/local endpoint.
- Conversation history.
- A reset button or equivalent new-chat functionality.

The developed Nexa Chatbot extended these basic requirements with additional chat-management functionality.

6.4 Technologies Used in Task 1

The following technologies were used specifically for Task 1:

Technology	Role in the Project
Python	Application logic and communication with the LLM
Streamlit	User interface and chatbot frontend
Ollama	Local LLM runtime
Llama 3.2	Locally hosted language model
JSON	Persistent chat-history storage

Technology	Role in the Project
Local API communication	Communication between the application and Ollama

The combination of these technologies allowed the project to run the chatbot locally and provide a browser-based interface for interacting with the model.

6.5 System Architecture

The architecture of the Task 1 application consists of three major components:

1. Frontend Layer

The frontend was developed using **Streamlit**.

It provides the interface through which the user can:

- Enter a prompt.
- View the assistant response.
- Access previous conversations.
- Start a new chat.
- Manage conversations.

2. Application Layer

Python handles the application logic.

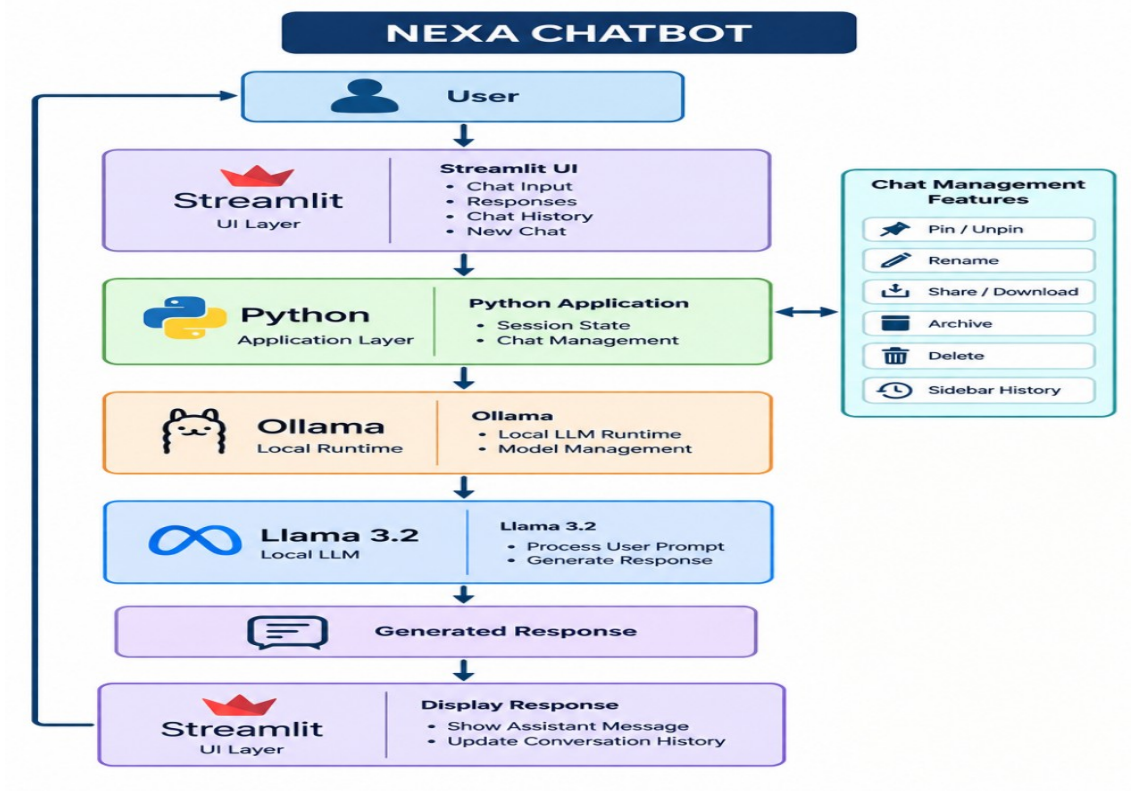
When a user submits a prompt, the application maintains the conversation messages and sends them to the local LLM through Ollama.

3. LLM Layer

Ollama provides the local runtime through which the **Llama 3.2** model is accessed.

The overall architecture can be represented as:

Figure 6.1: System Architecture of Nexa Chatbot



6.6 Environment Setup

The development environment for Task 1 consisted of a Windows-based local system with Python, Streamlit, Ollama, and the required language model.

The general setup process was:

Python Installation → Create/Configure Project Environment → Install Streamlit → Install Ollama → Download Llama 3.2 Model → Develop Python Application → Run Streamlit Application → Interact with Local LLM

The application was developed and tested locally through the Streamlit development server.

6.7 Ollama Setup and Local LLM

Ollama was used to run the Large Language Model locally.

The model selected for the chatbot was: llama3.2

Once Ollama was installed and the model was available locally, the Python application could communicate with it using the Ollama Python interface.

The key idea behind this setup was that the model inference was performed through the local Ollama environment rather than requiring a remote LLM API for every chatbot request.

6.7.1 Model Configuration

The model name was stored in the application as:

```
MODEL_NAME = "llama3.2"
```

This allowed the application to specify which locally available model should process the user's conversation.

6.7.2 Communication with Ollama

The application used the following basic logic to send the conversation to the model:

```
response = ollama.chat(
    model=MODEL_NAME,
    messages=st.session_state.messages
)

assistant_message = response["message"]["content"]
```

The process can be explained as:

1. The user enters a message.
2. The message is added to the conversation messages.
3. The messages are passed to Ollama.
4. Ollama sends the conversation to the local Llama 3.2 model.
5. The model generates a response.
6. The generated content is extracted from the response.
7. The response is displayed in the Streamlit interface.
8. The response is retained as part of the conversation.

This forms the core inference mechanism of the Nexa Chatbot.

6.8 Streamlit Application Development

Streamlit was used to convert the Python-based LLM functionality into an interactive web application.

The application provides a conversational interface where users can interact with the local model without directly using the command line.

The main interface components include:

- Application title and branding

- Chat interface
- User message area
- Assistant response area
- Sidebar
- Chat history
- New Chat functionality
- Chat-management controls
- File/image upload interface
- Additional interface controls

The application was designed to provide a chatbot experience while keeping the actual LLM inference local.

6.9 Task 1 Workflow

The complete workflow of the chatbot can be described as follows:

Step 1 – User enters a prompt

The user enters a question or instruction through the Nexa Chatbot interface.

Step 2 – Message is stored

The user message is added to the current conversation.

Step 3 – Application sends messages to Ollama

The complete conversation stored in the Streamlit session state is passed to the Ollama interface.

Step 4 – Llama 3.2 generates a response

Ollama uses the locally hosted Llama 3.2 model to generate the response.

Step 5 – Response is received

The generated assistant response is returned to the Python application.

Step 6 – Response is displayed

Streamlit displays the generated response in the chatbot interface.

Step 7 – Conversation is maintained

The user and assistant messages remain available as part of the current conversation.

User Prompt → Store User Message → Send Conversation to Ollama → Llama 3.2 Inference → Receive Assistant Response → Display Response → Update Conversation History

6.10 Conversation History

Conversation history was implemented so that users could continue a conversation instead of treating every prompt as an independent request.

The application maintains messages using Streamlit session state.

The conversation messages are stored in:

```
st.session_state.messages
```

The message history contains the conversation between the user and the assistant.

This allows previous messages to be supplied to the model when generating subsequent responses.

The application also uses:

```
chat_history.json
```

for persistent chat-history storage.

This provides a distinction between the **current active conversation** and the **stored chat history** that can be accessed through the application's chat-management interface.

6.11 New Chat / Reset Functionality

The application provides a **New Chat** functionality that allows the user to start a fresh conversation.

When a new chat is started, the active conversation can be cleared or reset so that the user can begin a new interaction without continuing the previous conversation.

This functionality addresses the reset requirement specified in the internship assignment.

6.12 Additional Chat Management Features

After implementing the basic chatbot functionality, additional chat-management features were incorporated into the **Nexa Chatbot** to improve conversation organization and user experience.

These features allow users to manage their previous conversations directly from the chatbot interface.

6.12.1 Chat History Sidebar

The application provides a sidebar containing the user's previous conversations.

The sidebar allows users to:

- View previous chats.
- Select a previous conversation.
- Access chat-management options.
- Organize conversations.
- Identify pinned and archived conversations.

The chat history is maintained using the application's chat-history storage mechanism.

6.12.2 Pin and Unpin Chat

The **Pin/Unpin** functionality allows users to organize important conversations in the chat history. Users can pin a conversation so that it remains easily accessible and prioritized in the chat list. When the conversation is no longer important, it can be unpinned.

6.12.3 Rename Chat

The **Rename Chat** functionality allows users to change the name of a conversation to a meaningful title. This makes it easier to identify and access specific conversations from the chat-history sidebar.

6.12.4 Share / Download

The **Share/Download** functionality provides options for users to preserve or share conversation-related content. This allows useful chatbot conversations or generated information to be retained for later reference or made available outside the application.

6.12.5 Archive Chat

The **Archive** functionality allows users to move conversations out of the primary active chat list without permanently deleting them. This helps keep the main chat-history area organized while retaining older conversations for future access.

6.12.6 Delete Chat

The **Delete** functionality allows users to remove conversations that are no longer required from the chat history. This gives users control over managing and removing unwanted conversations from the application.

6.13 File and Image Upload Interface

The Nexa Chatbot interface also contains an additional input-control area that provides options for working with files and images.

The interface includes options such as:

- File upload
- Image upload
- Image generation
- Browse Libraries

These options were incorporated into the chatbot interface to provide an extensible interaction area beyond simple text-based conversations.

Important: In the report, describe these as **interface features** unless your final implementation actually processes the uploaded files/images or generates images. This avoids claiming functionality that was not implemented.

Figure 6.2: File and Image Interaction Interface

You can insert a screenshot of the Nexa Chatbot showing the **plus (+) menu and its available options** here.

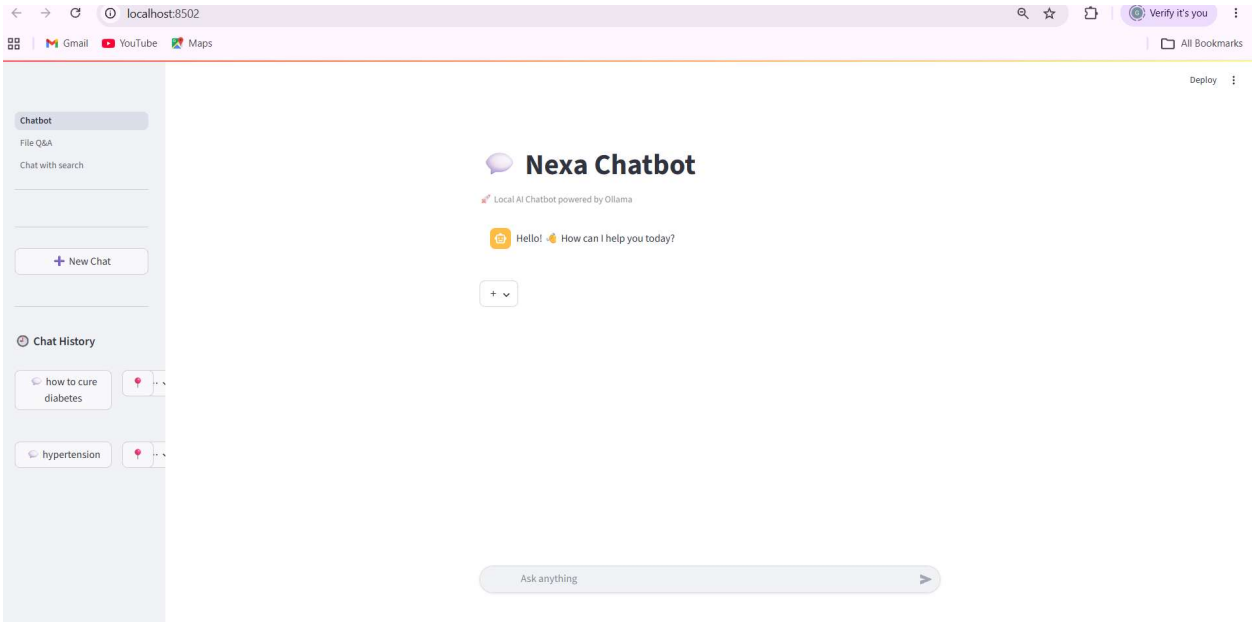


Figure 6.2: Nexa Chatbot

6.14 Application Session State

Streamlit session state was used to maintain information required during an active chatbot session.

Important session-state information included:

```
st.session_state.messages
```

This stores the messages belonging to the current conversation.

The application also maintains information related to:

- Current chat
- Chat history
- Conversation messages
- Chat-management operations

Using session state allows the application to maintain the current conversation while the user interacts with the Streamlit interface.

6.15 Code Implementation

The core inference implementation of the chatbot uses the Ollama Python interface.

A simplified version of the implementation is:

```
response = ollama.chat(
    model=MODEL_NAME,
    messages=st.session_state.messages
)
assistant_message = response["message"]["content"]
```

6.16 Working Demonstration

The working of the Nexa Chatbot can be summarized in the following sequence:

Step 1 – Start the application

The Streamlit application is launched locally.

Step 2 – Open Nexa Chatbot

The chatbot interface is displayed in the browser.

Step 3 – Enter a prompt

The user enters a question or instruction into the chat input.

Step 4 – Send the prompt

The user's message is added to the current conversation.

Step 5 – Send request to Ollama

The Python application passes the conversation messages to Ollama.

Step 6 – Local model inference

Ollama uses the locally hosted **Llama 3.2** model to process the request.

Step 7 – Generate response

The model generates an appropriate text response.

Step 8 – Display response

The response is returned to the Streamlit application and displayed to the user.

Step 9 – Maintain conversation

The user and assistant messages are retained in the conversation history.

Step 10 – Manage conversation

The user can subsequently start a new chat or manage existing conversations using the available chat-management options.

6.17 Task 1 Results

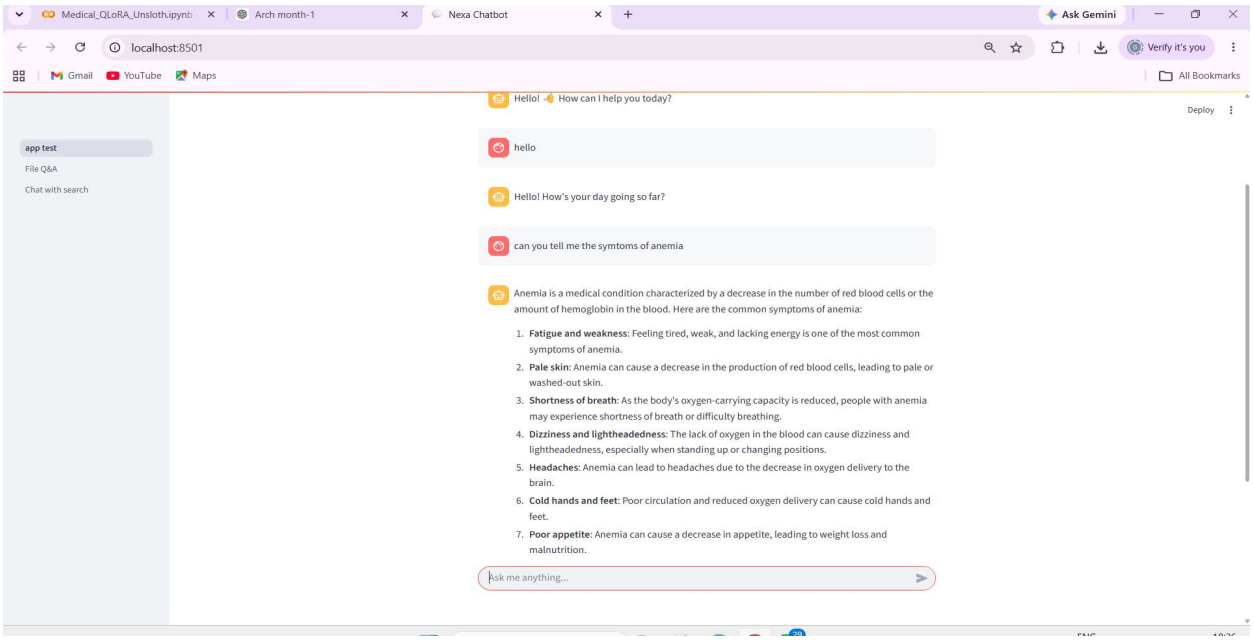
The implementation of Task 1 resulted in a functional Streamlit-based chatbot application named **Nexa Chatbot**.

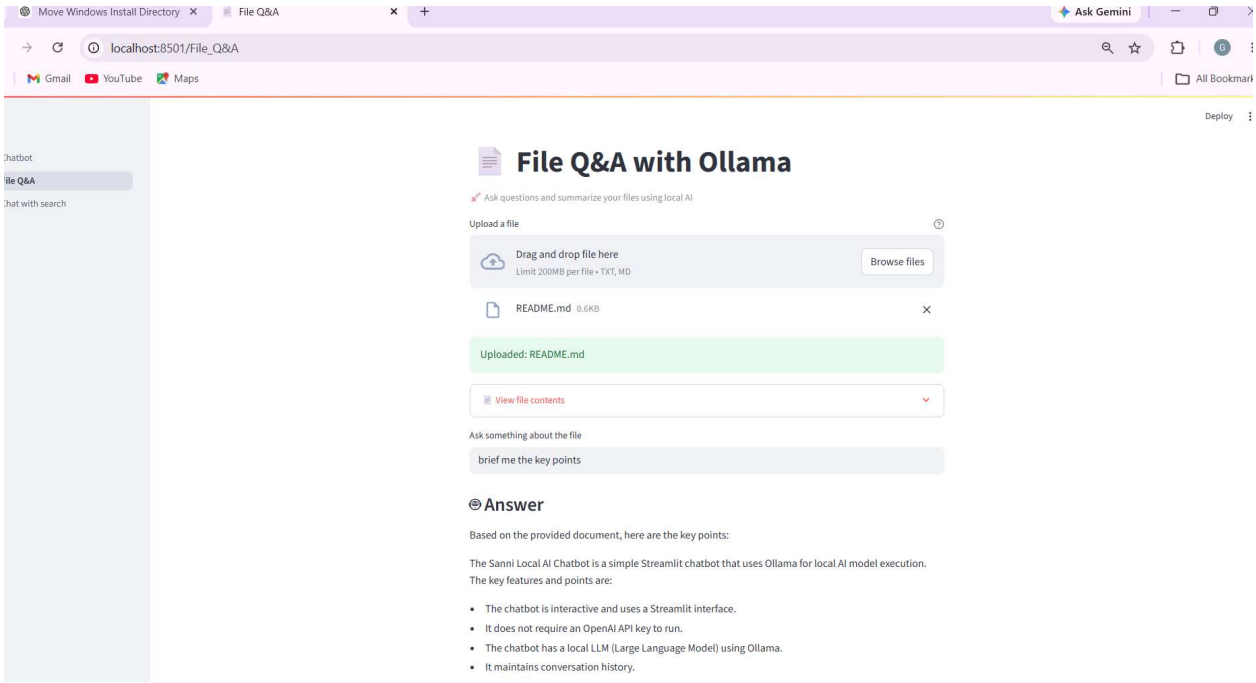
The completed implementation achieved the main requirements of the assigned task:

Requirement	Implementation
Streamlit interface	Implemented
Local LLM	Llama 3.2
Local LLM runtime	Ollama
Text-based interaction	Implemented
Model response display	Implemented
Conversation history	Implemented
New chat functionality	Implemented
Chat management	Implemented
Persistent chat history	Implemented using JSON
Additional interface controls	Implemented

The project provided practical experience in connecting a locally hosted Large Language Model with a web-based application.

6.18 Task 1 Screenshots


Medical_QLoRA_Unsloth.ipynb
Arch month-1
Nexa Chatbot
Ask Gemini
localhost:8501
Verify it's you
Gmail
YouTube
Maps
All Bookmarks
Deploy
app test
File Q&A
Chat with search
Hello! How can I help you today?
hello
Hello! How's your day going so far?
can you tell me the symtoms of anemia
Anemia is a medical condition characterized by a decrease in the number of red blood cells or the amount of hemoglobin in the blood. Here are the common symptoms of anemia:
1. Fatigue and weakness: Feeling tired, weak, and lacking energy is one of the most common symptoms of anemia.
2. Pale skin: Anemia can cause a decrease in the production of red blood cells, leading to pale or washed-out skin.
3. Shortness of breath: As the body's oxygen-carrying capacity is reduced, people with anemia may experience shortness of breath or difficulty breathing.
4. Dizziness and lightheadedness: The lack of oxygen in the blood can cause dizziness and lightheadedness, especially when standing up or changing positions.
5. Headaches: Anemia can lead to headaches due to the decrease in oxygen delivery to the brain.
6. Cold hands and feet: Poor circulation and reduced oxygen delivery can cause cold hands and feet.
7. Poor appetite: Anemia can cause a decrease in appetite, leading to weight loss and malnutrition.
Ask me anything...


Move Windows Install Directory
File Q&A
Ask Gemini
localhost:8501/File_Q&A
Gmail
YouTube
Maps
Deploy
File Q&A with Ollama
Ask questions and summarize your files using local AI
Upload a file
Drag and drop file here
Limit 200MB per file • TXT, MD
Browse files
README.md 0.6KB
Uploaded: README.md
View file contents
Ask something about the file
brief me the key points
Answer
Based on the provided document, here are the key points:
The Sanni Local AI Chatbot is a simple Streamlit chatbot that uses Ollama for local AI model execution. The key features and points are:
- The chatbot is interactive and uses a Streamlit interface.
- It does not require an OpenAI API key to run.
- The chatbot has a local LLM (Large Language Model) using Ollama.
- It maintains conversation history.

TASK 2 – MEDICAL FINE-TUNING WITH QUANTIZATION AND LOW-RANK ADAPTATION USING UNSLOTH

The internship assignment required implementing a QLoRA-based fine-tuning workflow using a domain-specific medical dataset, 4-bit quantization, LoRA adapter configuration, training, monitoring, saving the adapter, and testing the resulting model with new medical queries.

7.1 Task 2 Objective

The objective of Task 2 was to understand and implement an efficient Large Language Model fine-tuning workflow for a medical domain**.**

The task involved using Unsloth in Google Colab to perform parameter-efficient fine-tuning using QLoRA. The workflow included loading a pretrained language model, preparing a medical question-answering dataset, applying 4-bit quantization, configuring LoRA adapters, training the model, saving the trained adapter, and testing the model using new medical queries.

The main objective was to understand how a pretrained LLM can be adapted to a specialized domain without requiring the complete model to be trained from scratch.

7.2 Problem Statement

Large Language Models are generally trained on broad datasets and may not be specifically optimized for a particular domain. Fine-tuning allows a pretrained model to learn patterns from a specialized dataset and become more suitable for a particular task or domain.

However, fine-tuning large language models can require considerable computational memory and processing resources.

To address this challenge, Task 2 used 4-bit quantization together with LoRA, forming a QLoRA-based fine-tuning workflow. Unsloth was used to make the training process more efficient within the Google Colab environment.

The medical domain was selected for the project, and a medical question-answering dataset was used for training.

7.3 Task Requirements

The internship assignment specified the following requirements for Task 2:

1. Implement a QLoRA-based fine-tuning workflow.
2. Use Unsloth in Google Colab.
3. Load a domain-specific medical dataset.
4. Configure 4-bit quantization.
5. Configure low-rank adaptation using LoRA.
6. Perform tokenization and dataset preparation.
7. Execute epoch-based model training.
8. Monitor memory usage and training performance.
9. Save the fine-tuned adapter.
10. Test the trained model using new medical queries.

The implemented project followed these requirements using a medical question-answering dataset and a Llama-based pretrained model.

7.4 Introduction to LLM Fine-Tuning

Fine-tuning is the process of taking a pretrained machine learning model and training it further using a dataset designed for a particular task or domain.

Instead of training a language model completely from the beginning, a pretrained model is used as the starting point. The model is then adapted using domain-specific examples.

In this project, the pretrained language model was adapted using medical question-and-answer examples.

The overall concept can be represented as:

Pretrained LLM → Medical Dataset → Fine-Tuning → Medical-Domain Adapted Model → Medical Query Testing

7.5 Quantization

Quantization is a technique used to represent model parameters using lower numerical precision.

Large language models can require significant memory when their parameters are stored using higher precision. Quantization reduces the memory requirement by using a lower-bit representation.

In this project, **4-bit quantization** was used while loading the base model.

The model was loaded using:

```
load_in_4bit=True
```

The use of 4-bit quantization helped make the model suitable for fine-tuning within the available Google Colab GPU environment.

7.6 Low-Rank Adaptation (LoRA)

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique.

Instead of updating all parameters of the pretrained model, LoRA introduces additional trainable low-rank matrices into selected model layers.

This reduces the number of parameters that need to be trained and can significantly reduce the computational and memory requirements of fine-tuning.

In this project, LoRA adapters were added to important projection layers of the Llama model.

The configured target modules were:

q_proj

k_proj

v_proj

o_proj

gate_proj

up_proj

down_proj

The LoRA configuration used a rank of **16** and an alpha value of **16**.

7.7 QLoRA

QLoRA combines **4-bit quantization** with **LoRA-based parameter-efficient fine-tuning**.

In the implemented workflow, the pretrained model was loaded using 4-bit quantization, while LoRA adapters were added for training.

The overall process was:

Pretrained Model

↓

4-bit Quantization

↓

LoRA Adapter Configuration

↓

QLoRA Fine-Tuning

↓

Save Adapter

↓

Test Model

This approach allowed the fine-tuning workflow to be performed using the available GPU resources in Google Colab.

The assignment specifically required a 4-bit quantized low-rank adaptation workflow.

7.8 Unsloth

Unsloth was used as the main framework for the efficient fine-tuning workflow.

It was used for tasks such as:

- Loading the pretrained model.

- Configuring 4-bit model loading.
- Setting up LoRA adapters.
- Preparing the model for parameter-efficient training.
- Supporting the training workflow.
- Saving the trained adapter.

The use of Unsloth helped implement the required QLoRA workflow in Google Colab.

7.9 Google Colab Environment

Google Colab was used as the development and training environment for Task 2.

The environment provided access to GPU acceleration, which was required for the model fine-tuning process.

The training was performed using an **NVIDIA Tesla T4 GPU**.

The general environment was:

Google Colab

↓

NVIDIA Tesla T4 GPU

↓

Python

↓

Unsloth

↓

Llama 3.2 1B Instruct

↓

QLoRA Fine-Tuning

The use of a GPU significantly improves the practicality of training and fine-tuning neural network models compared with CPU-only execution.

7.10 Base Model

The pretrained model selected for the project was:

```
unsloth/Llama-3.2-1B-Instruct
```

This model was used as the starting point for the medical-domain fine-tuning process.

The model was loaded using Unsloth with 4-bit quantization enabled.

The basic loading configuration included:

```
load_in_4bit=True
```

The pretrained model was not trained completely from scratch. Instead, the existing language model was adapted using the medical question-answering dataset.

7.11 Medical Dataset

The medical dataset used for Task 2 was **MedInstruct-52k**, obtained from the Hugging Face dataset repository. MedInstruct-52k is a medical instruction-following dataset containing medical instructions and corresponding responses. Unlike a traditional question-answer dataset, the data is structured to support instruction-based interaction between the user and the language model.

The dataset was downloaded as a JSON file using the following command:

```
https://huggingface.co/datasets/casey-
martin/MedInstruct/resolve/main/data/MedInstruct-52k.json
```

The downloaded dataset was loaded into the Google Colab environment and prepared for fine-tuning. The instruction and response information was converted into a conversational format compatible with the Llama 3.2 1B Instruct model.

The use of MedInstruct-52k allowed the fine-tuning process to focus on medical-domain instruction following rather than only direct question-and-answer pairs.

7.12 Dataset Preparation

The MedInstruct-52k dataset was prepared before beginning the fine-tuning process. Since the downloaded dataset was provided in JSON format, the data was first loaded into the Google Colab environment and its instruction-response structure was examined.

The preparation process involved loading the JSON dataset, extracting the relevant instruction and response information, organizing the examples into a conversational structure, and applying the appropriate Llama chat template.

The general workflow was:

MedInstruct-52k JSON Dataset

↓

Load Dataset

↓

Inspect Instruction-Response Data

↓

Prepare Training Examples

↓

Format as Conversation

↓

Apply Llama Chat Template

↓

Training Dataset

The preparation of the dataset was important because the language model needs training examples in a format that represents how users and assistants interact.

7.13 Dataset Formatting Using Chat Template

The question-and-answer pairs were converted into a conversational format using the Llama chat template.

A typical conversational structure contains:

```
User:
Medical Question
```

```
Assistant:
Medical Answer
```

The chat template converts this information into the format expected by the instruction-tuned language model.

This allows the model to learn the relationship between a medical question and its corresponding answer during fine-tuning.

7.14 4-Bit Quantization Configuration

The pretrained model was loaded using 4-bit quantization.

The main configuration was:

```
load_in_4bit=True
```

This configuration reduced the memory requirements of the model during the fine-tuning process.

The use of 4-bit quantization was an important part of the QLoRA workflow required by the internship assignment.

7.15 LoRA Adapter Configuration

After loading the model, LoRA adapters were configured using Unsloth.

The implementation used:

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ],
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
    use_gradient_checkpointing="unsloth",
    random_state=3407,
)
```

Parameter	Value
LoRA Rank (`r`)	16

Parameter	Value
LoRA Alpha	16
LoRA Dropout	0
Bias	None
Gradient Checkpointing	Unsloth
Random State	3407

The selected target modules correspond to important projection layers within the transformer architecture.

7.16 Training Configuration

After preparing the dataset and configuring the LoRA adapter, the model was prepared for supervised fine-tuning.

The training workflow included:

```
Prepared Dataset
       ↓
Tokenization
       ↓
LoRA-Enabled Model
       ↓
SFTTrainer
       ↓
Epoch-Based Training
       ↓
Training Metrics
```

The **SFTTrainer** was used for supervised fine-tuning of the model using the prepared medical question-answering examples.

7.17 Model Fine-Tuning

The fine-tuning process was executed in Google Colab using the Tesla T4 GPU.

During training, the model learned from the selected medical question-answer pairs. Because QLoRA was used, the pretrained model was kept in a quantized representation while the LoRA parameters were trained.

The overall training process was:

```
Llama 3.2 1B Instruct
          ↓
```

```
4-bit Quantization
        ↓
LoRA Adapter
        ↓
Medical QA Training Data
        ↓
SFTTrainer
        ↓
Training
        ↓
Fine-Tuned Adapter
```

The training process allowed the pretrained model to be adapted toward the medical question-answering domain.

7.18 Training Monitoring

The training process was monitored through the Google Colab environment.

The training output provided information about the progress and performance of the training process.

Important aspects monitored during training included:

- Training progress
- Training steps
- Training loss
- GPU memory usage
- Training performance
- Completion of the training process

Monitoring these values helped verify that the fine-tuning process was running successfully within the available computational resources.

7.19 Saving the Fine-Tuned Adapter

After training was completed, the trained LoRA adapter was saved.

Saving the adapter is useful because the complete pretrained model does not need to be saved again for every fine-tuning task. The adapter contains the learned modifications produced during the domain-specific training process.

The workflow was:

```
Base Model
    +
Trained LoRA Adapter
    ↓
Fine-Tuned Model Configuration
```

The saved adapter could subsequently be loaded for inference and testing.

7.20 Testing the Fine-Tuned Model

After training and saving the adapter, the model was tested using new medical queries.

The purpose of testing was to verify whether the fine-tuned model could generate responses for medical questions that were not directly part of the training examples.

The testing workflow was:

```
New Medical Query
        ↓
Fine-Tuned Model
        ↓
Model Inference
        ↓
Generated Medical Response
```

This step completed the basic QLoRA fine-tuning workflow required by the assignment. The assignment specifically required testing the fine-tuned model with new medical queries.

7.21 Task 2 Workflow Summary

The complete implementation can be represented as:

7.22 Task 2 Results

The implementation of Task 2 provided practical experience with an end-to-end parameter-efficient fine-tuning workflow.

The completed workflow included:

Requirement	Implementation
Google Colab	Used
GPU environment	Tesla T4
Base model	Llama 3.2 1B Instruct
Medical dataset	MedInstruct-52k
Training examples	2,000 selected examples
Quantization	4-bit
Fine-tuning method	LoRA / QLoRA
Fine-tuning framework	Unsloth
Training method	Supervised fine-tuning

Requirement	Implementation
Adapter	Trained and saved
Model testing	Tested using new medical queries

The task demonstrated how a pretrained LLM can be adapted to a specialized medical question-answering domain using a memory-efficient QLoRA workflow.

7.23 Task 2 Screenshots

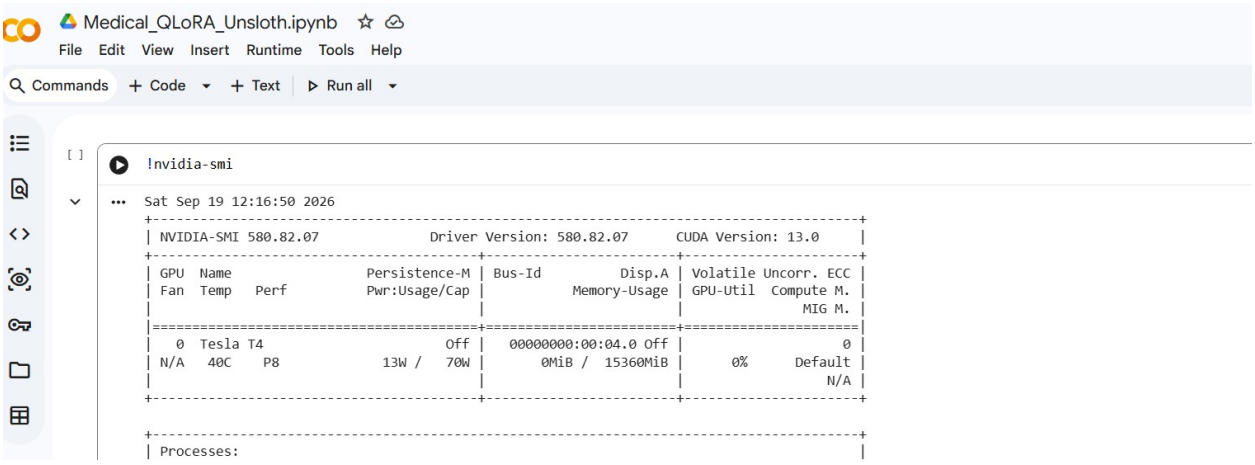


Figure 7.1: Google Colab Environment with Tesla T4 GPU

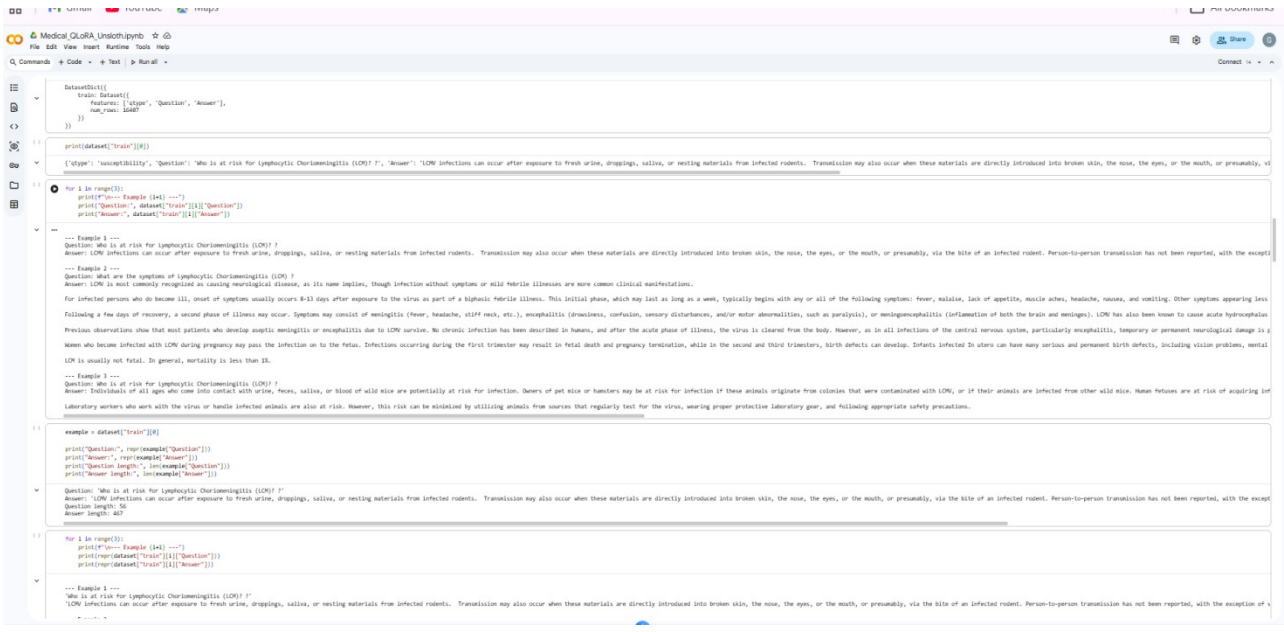


Figure 7.2: Loading the Medical Question-Answering Dataset

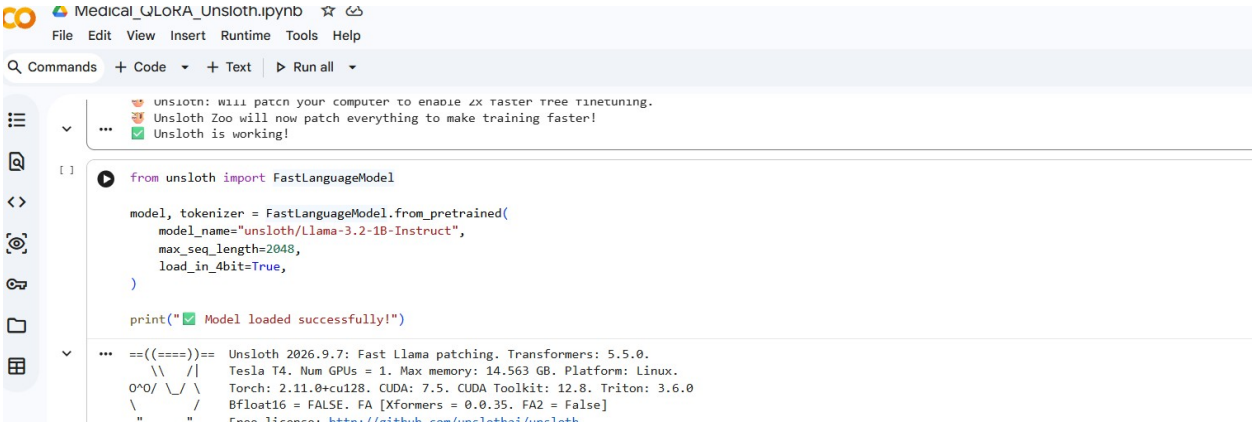


Figure 7.3 – Loading the Llama 3.2 1B Instruct Model with 4-bit Quantization

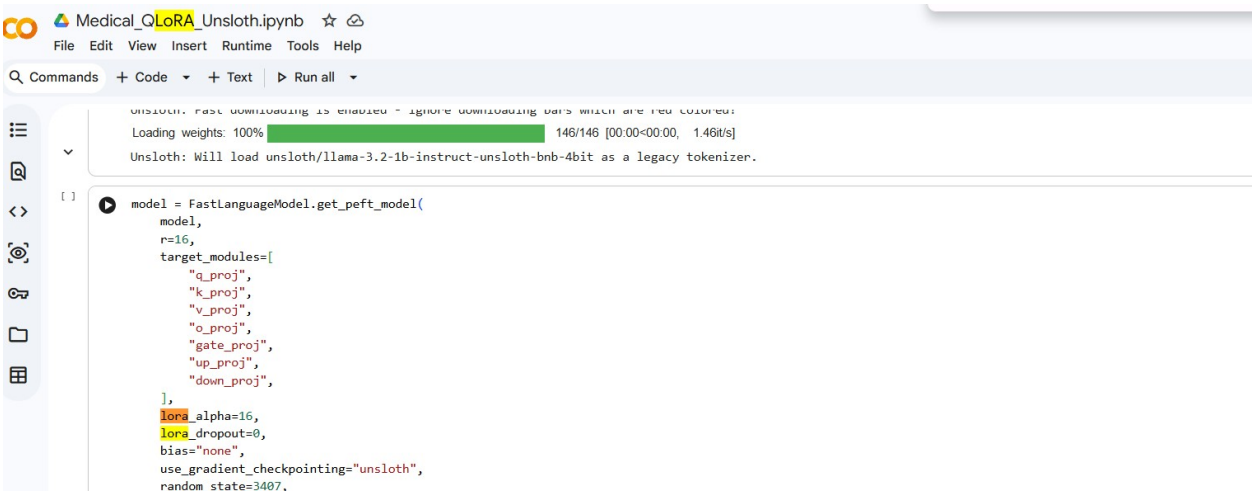


Figure 7.4: LoRA Adapter Configuration

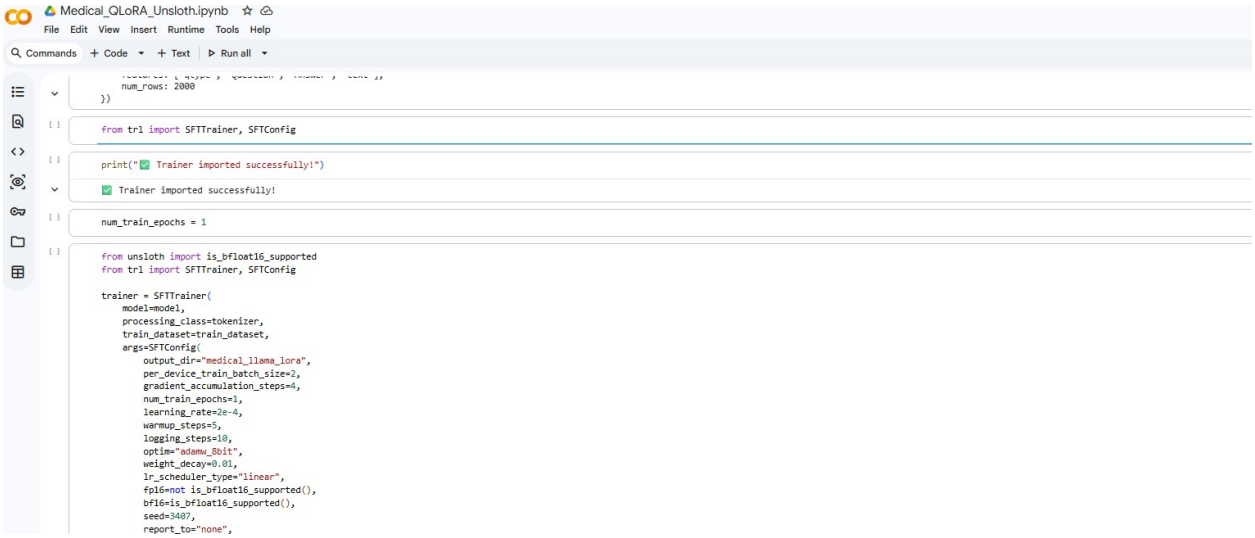
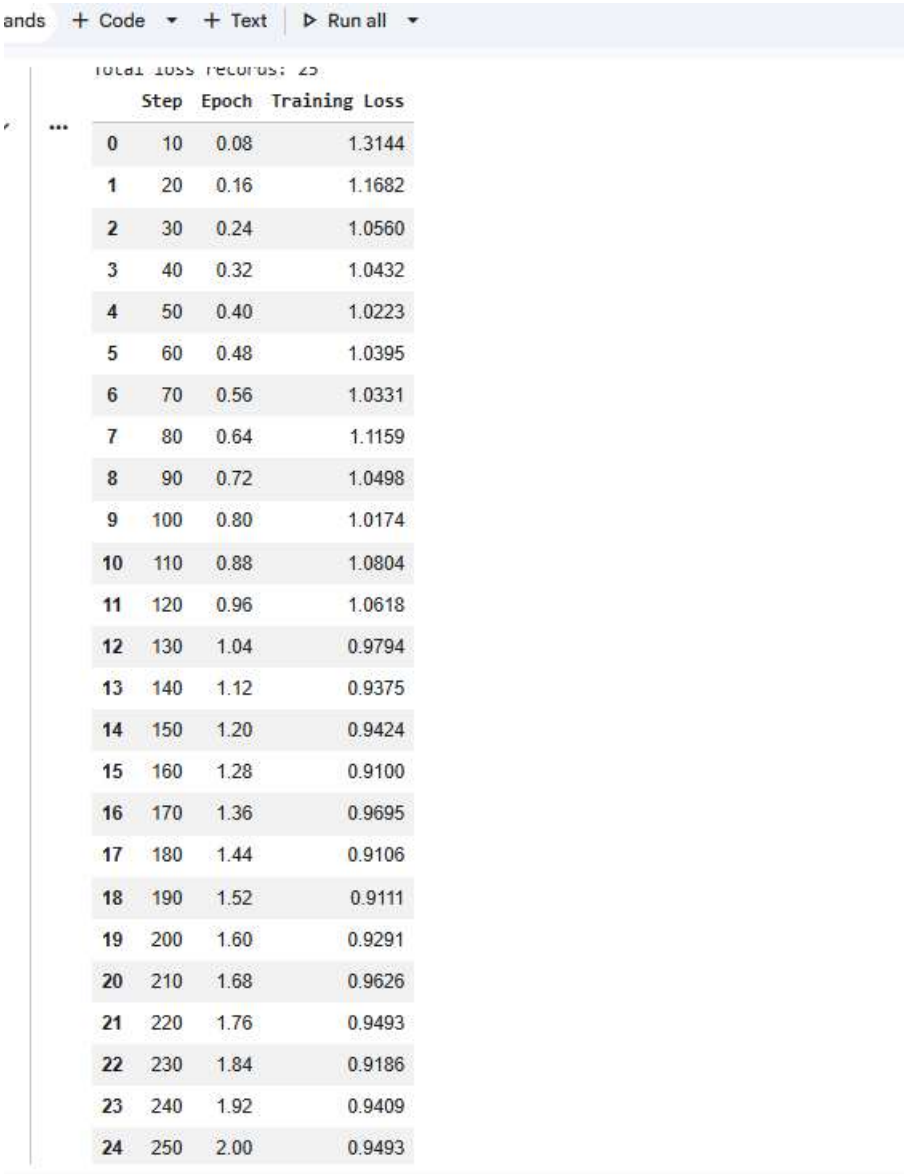


Figure 7.5: Medical Domain Fine-Tuning Using QLoRA



	Step	Epoch	Training Loss
0	10	0.08	1.3144
1	20	0.16	1.1682
2	30	0.24	1.0560
3	40	0.32	1.0432
4	50	0.40	1.0223
5	60	0.48	1.0395
6	70	0.56	1.0331
7	80	0.64	1.1159
8	90	0.72	1.0498
9	100	0.80	1.0174
10	110	0.88	1.0804
11	120	0.96	1.0618
12	130	1.04	0.9794
13	140	1.12	0.9375
14	150	1.20	0.9424
15	160	1.28	0.9100
16	170	1.36	0.9695
17	180	1.44	0.9106
18	190	1.52	0.9111
19	200	1.60	0.9291
20	210	1.68	0.9626
21	220	1.76	0.9493
22	230	1.84	0.9186
23	240	1.92	0.9409
24	250	2.00	0.9493

Figure 7.6: Training Progress and Performance Monitoring



Figure 7.7: Saving the Fine-Tuned LoRA Adapter

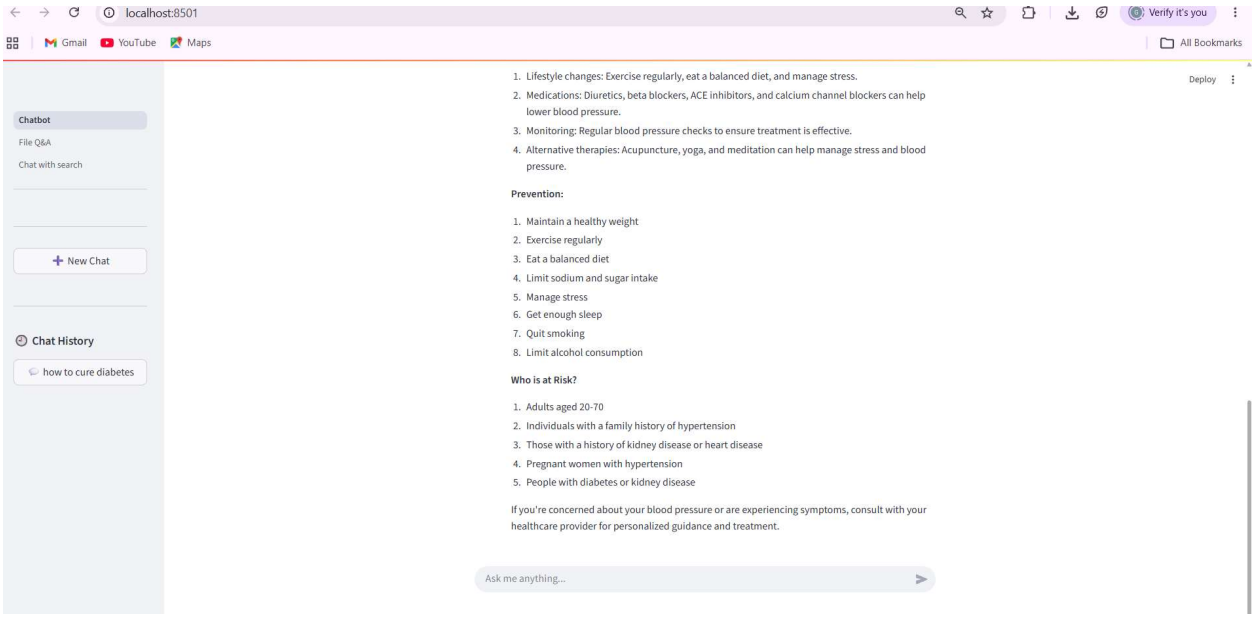


Figure 7.8: Testing the Fine-Tuned Model with a New Medical Query

INTEGRATION OF TASK 1 AND TASK 2

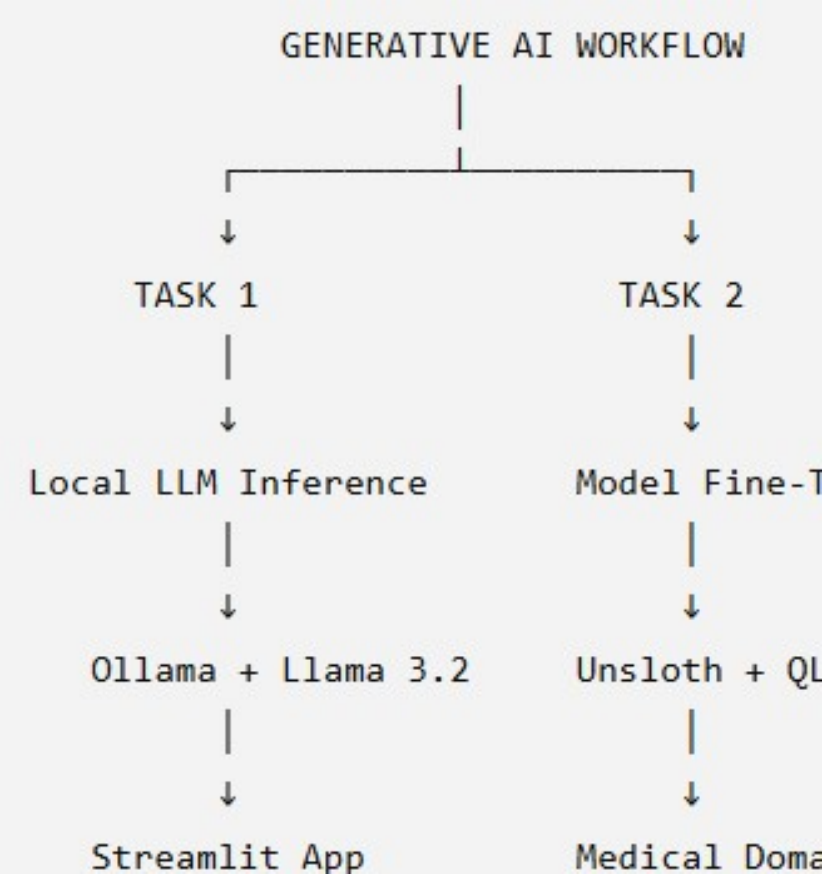
8.1 Integration Overview

The Month 1 internship consisted of two major components involving Large Language Models.

Task 1 focused on the application and inference side of Generative AI. A Streamlit-based chatbot named Nexa Chatbot was developed to interact with a locally hosted Llama 3.2 model through Ollama.

Task 2 focused on the model adaptation and fine-tuning side. A Llama 3.2 1B Instruct model was fine-tuned for the medical question-answering domain using 4-bit quantization, LoRA, QLoRA, and Unsloth in Google Colab.

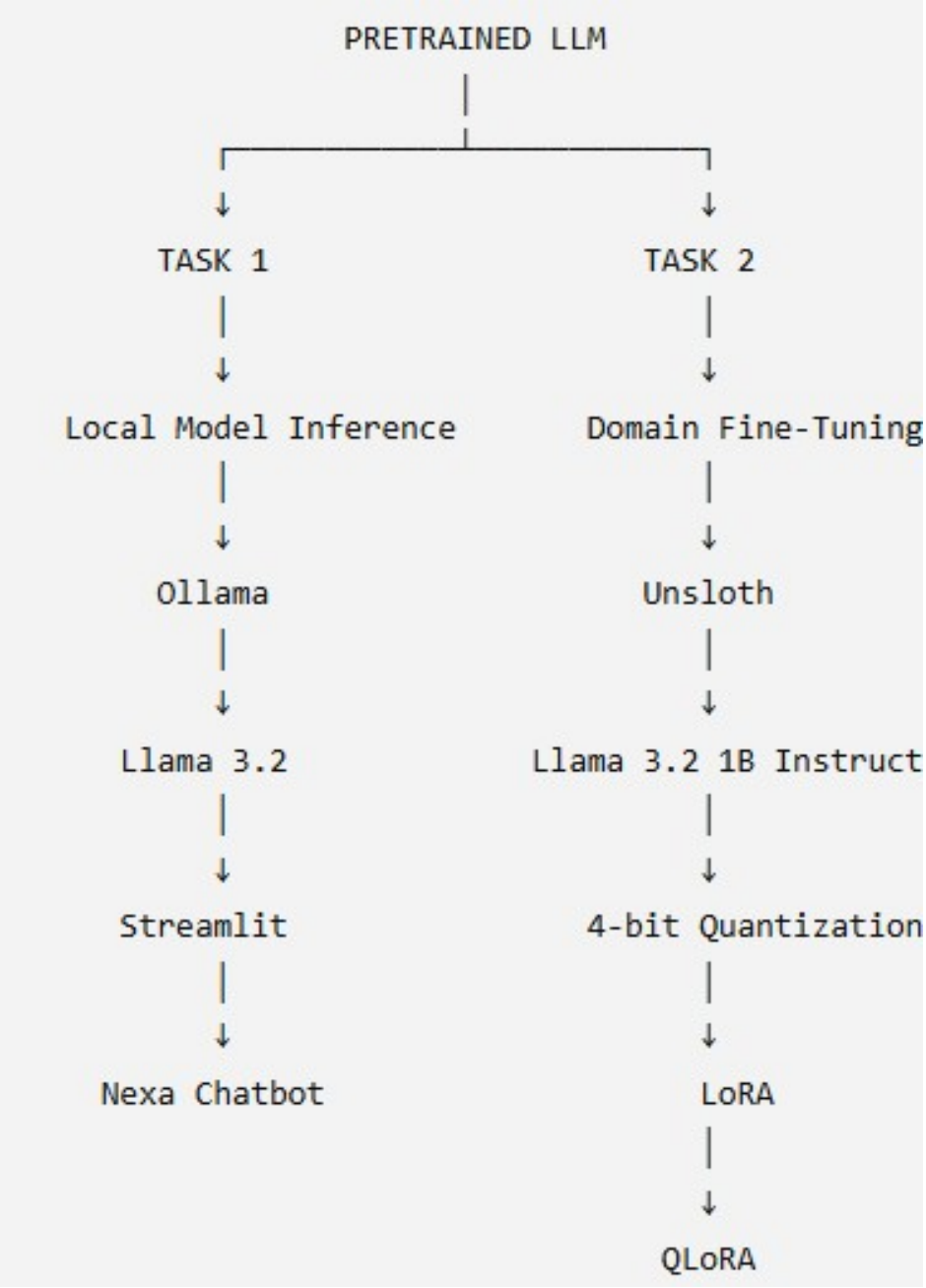
Therefore, the two tasks represent two complementary stages:



8.2 End-to-End Workflow

The overall Month 1 workflow began with understanding how a pretrained LLM can be used in an application and then progressed to understanding how the model can be adapted for a specialized domain.

The workflow can be summarized as:



The two tasks therefore provided practical exposure to both **LLM application development** and **LLM fine-tuning**.

8.3 Model Inference Workflow

In Task 1, the inference workflow starts when a user enters a prompt into the Nexa Chatbot.

The application sends the conversation messages to Ollama, which uses the locally hosted Llama 3.2 model to generate a response.

```
User
 ↓
Nexa Chatbot
 ↓
Streamlit
 ↓
Python Application
 ↓
Ollama
 ↓
Llama 3.2
 ↓
Generated Response
 ↓
```

This workflow demonstrates how a locally hosted LLM can be integrated into a user-facing application.

8.4 Fine-Tuning Workflow

Task 2 followed a different workflow because its purpose was to adapt a pretrained model to the medical domain.

The process was:

Pretrained Llama Model

↓

Medical QA Dataset

↓

Dataset Preparation

↓

4-bit Quantization

↓

LoRA Adapter

↓

QLoRA Fine-Tuning

↓

Saved Adapter

↓

Medical Query Testing

This demonstrated how parameter-efficient techniques can be used to adapt a pretrained model using domain-specific data.

8.5 Relationship Between Task 1 and Task 2

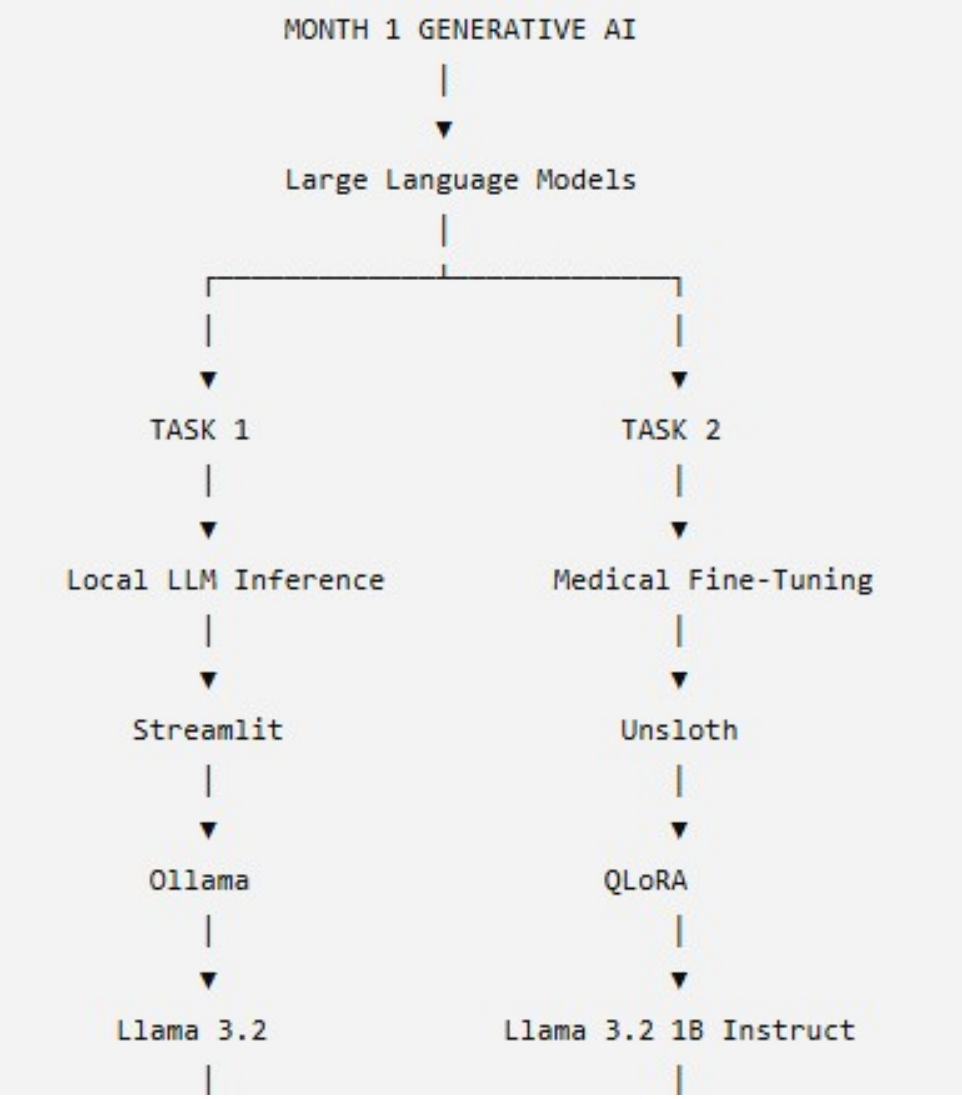
Task	Main Focus	Technologies
Task 1	LLM application and local inference	Streamlit, Ollama, Llama 3.2
Task 2	LLM domain adaptation and fine-tuning	Google Colab, Unsloth, LoRA, QLoRA
Task 1 Output	Nexa Chatbot	Local conversational application
Task 2 Output	Fine-tuned LoRA adapter	Medical-domain adapted model

Task 1 provided practical experience in **using an LLM**, while Task 2 provided practical experience in **adapting an LLM**.

Together, they helped provide an understanding of the broader LLM development workflow.

8.6 Overall Month 1 Architecture

The overall learning and implementation workflow of Month 1 can be represented as:



8.7 Integration Outcome

The completion of both tasks provided practical exposure to two important aspects of Generative AI development.

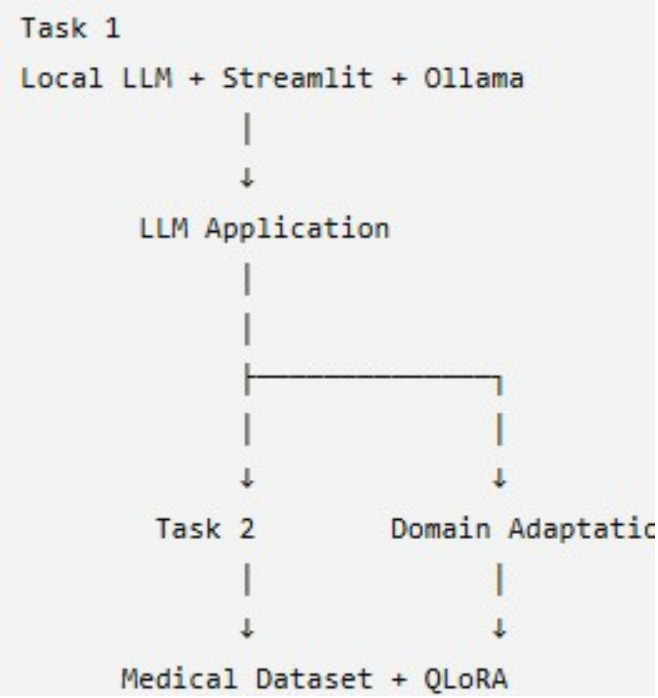
Through **Task 1**, a locally hosted LLM was connected to a Streamlit application and presented through the Nexa Chatbot interface.

Through **Task 2**, the process of adapting a pretrained language model to a medical question-answering domain was implemented using an efficient QLoRA-based fine-tuning workflow.

The two tasks together provided experience ranging from LLM inference and application development to domain-specific model fine-tuning and testing**.**

8.8 Summary of Integration

The Month 1 project can therefore be summarized as:



This completed the integration and overall workflow documentation for the two major tasks performed during Month 1.

CODE IMPLEMENTATION

9.1 Task 1 – Streamlit Application Code

The first task involved developing a Streamlit-based chatbot interface that communicates with a locally hosted LLM through Ollama.

The application was developed using Python and Streamlit. The main responsibilities of the code were:

- Creating the chatbot interface.
- Accepting user messages.
- Maintaining conversation messages using Streamlit session state.
- Sending the conversation to the locally hosted Llama 3.2 model.
- Receiving the model-generated response.
- Displaying the response in the chatbot interface.
- Maintaining persistent chat history using chat_history.json.
- Providing chat-management operations such as creating, renaming, pinning, archiving, and deleting conversations.

9.1.1 Model Configuration

The locally hosted model used in the application was **Llama 3.2**, accessed through Ollama.

```
MODEL_NAME = "llama3.2"
```

The model name was stored in a variable so that it could be easily referenced when making requests to Ollama.

9.1.2 Sending Messages to Ollama

The main communication between the Streamlit application and the locally hosted LLM was implemented using the Ollama Python interface.

```
response = ollama.chat(
    model=MODEL_NAME,
    messages=st.session_state.messages
)

assistant_message = response["message"]["content"]
```

The ollama.chat() function sends the conversation messages to the selected local model. The model processes the conversation and generates a response. The generated text is then extracted from the response and displayed to the user.

9.1.3 Session State for Conversation

Streamlit session state was used to maintain the current conversation while the application was running.

```
if "messages" not in st.session_state:
    st.session_state.messages = []
```

The conversation messages are stored in st.session_state.messages. This allows the application to maintain the context of the current conversation when the Streamlit interface is updated.

9.1.4 User and Assistant Messages

When a user enters a message, it is added to the conversation before the request is sent to Ollama.

The general workflow is:

User Input

↓

Add User Message

↓

Send Conversation to Ollama

↓

Generate LLM Response

↓

Add Assistant Response

↓

Display Response

This approach allows the LLM to receive the previous conversation messages along with the latest user prompt.

9.1.5 Persistent Chat History

The application also used a JSON file named:

```
chat_history.json
```

The file was used to preserve conversations beyond the current Streamlit session. This enabled the application to provide a chat-history sidebar where previous conversations could be accessed and managed.

9.1.6 Chat Management

Additional Python logic was implemented for managing conversations. The application provided options such as:

- New Chat
- Pin / Unpin
- Rename
- Share / Download
- Archive
- Delete

These features improved the usability of the chatbot by allowing users to organize their conversations.

9.2 Task 2 – Medical Fine-Tuning Code

The second task implemented a medical-domain fine-tuning workflow using **Unsloth, 4-bit quantization, LoRA, and QLoRA** in Google Colab.

The base model used was:

```
unsloth/Llama-3.2-1B-Instruct
```

The medical dataset used was:

```
MedInstruct-52k
```

A subset of 2,000 examples was selected from the available dataset for the training workflow.

9.2.1 Loading the Base Model

The model was loaded using Unsloth with 4-bit loading enabled.

The important configuration was:

```
load_in_4bit=True
```

Using 4-bit loading reduces the memory required to load the model and makes it more practical to perform fine-tuning in a Google Colab GPU environment.

9.2.2 Dataset Preparation

The medical question-answering dataset contained the following fields:

```
qtype
Question
Answer
```

The dataset was prepared so that each medical question and its corresponding answer could be converted into the conversational format expected by the instruction-tuned model.

A Llama chat template was used to format the examples.

The general structure was:

```
User:
Medical Question

Assistant:
Medical Answer
```

This formatting allows the training examples to resemble the type of interaction the model will receive during inference.

9.2.3 LoRA Adapter Configuration

The LoRA adapter was configured using the Unsloth `FastLanguageModel.get_peft_model()` function.

The implementation used the following configuration:

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=[
        "q_proj","k_proj","v_proj","o_proj",
        "gate_proj","up_proj","down_proj",
    ],
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
```

```
    use_gradient_checkpointing="unsloth",
    random_state=3407,
)
```

Explanation of Important Parameters

r=16

The rank of the LoRA adapter was set to 16. The rank determines the size of the additional trainable low-rank matrices used by LoRA.

target_modules

The LoRA adapters were applied to important projection layers of the transformer model, including:

```
q_proj
k_proj
v_proj
o_proj
gate_proj
up_proj
down_proj
```

These modules are associated with the attention and feed-forward components of the model.

lora_alpha=16

This parameter controls the scaling applied to the LoRA updates.

lora_dropout=0

No dropout was applied to the LoRA layers in this configuration.

bias="none"

Bias parameters were not separately trained as part of the LoRA configuration.

use_gradient_checkpointing="unsloth"

Unsloth's gradient-checkpointing configuration was enabled to help reduce memory usage during training.

9.2.4 Supervised Fine-Tuning

The prepared dataset was passed to an SFTTrainer for supervised fine-tuning.

The training process involved:

```
Prepared Medical Dataset
        ↓
Chat Template Formatting
        ↓
Tokenization
        ↓
LoRA-Configured Model
        ↓
SFT Training
        ↓
Fine-Tuned LoRA Adapter
```

During training, the model learned from the medical question-answer pairs while the LoRA adapter parameters were updated.

9.2.5 Saving the Adapter

After training was completed, the trained adapter was saved for later use.

The saved adapter represents the learned domain-specific modifications without requiring the complete base model to be stored as a newly trained model.

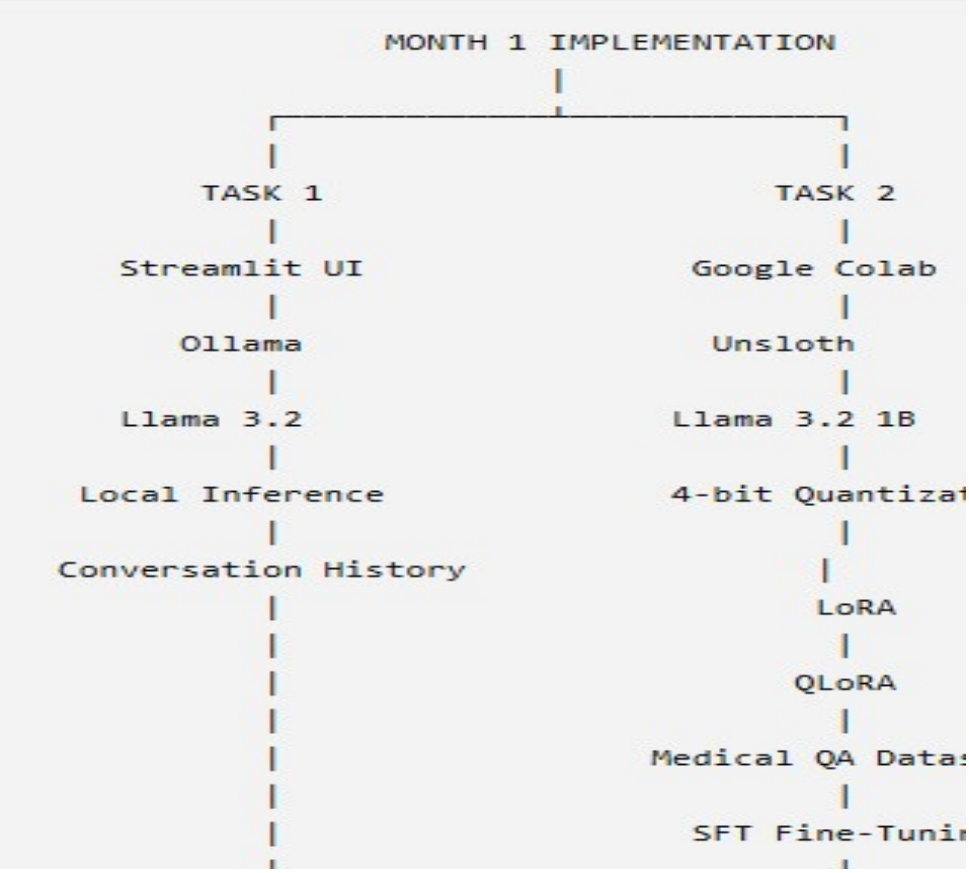
9.2.6 Testing the Fine-Tuned Model

After training and saving the adapter, the resulting model was tested using new medical questions.

The testing stage was used to verify that the fine-tuned model could generate responses for medical-domain queries after the training process.

9.3.1 Overall Code Workflow

The complete Month 1 implementation can be represented as:



9.3.2 Code Implementation Outcome

The code implementation successfully covered both major objectives of the Month 1 internship. Task 1 demonstrated how a locally hosted Large Language Model can be connected to an interactive web interface using Streamlit and Ollama. Task 2 demonstrated how a pretrained language model can be adapted to a medical question-answering domain using 4-bit quantization, LoRA, and QLoRA with Unsloth.

RESULTS AND OBSERVATIONS

The Month 1 internship project provided practical experience in two important areas of Generative AI: LLM application development and LLM fine-tuning**.** The results obtained from Task 1 and Task 2 are described below.

10.1 Task 1 Results

The first task resulted in the development of a functional Streamlit-based chatbot application named **Nexa Chatbot**.

The application successfully provided an interface through which users could interact with a locally hosted **Llama 3.2** model using Ollama.

The major results achieved were:

1. A functional Streamlit chatbot interface was developed.
2. The application was connected to a locally hosted LLM through Ollama.
3. Users could enter prompts and receive responses generated by the local model.
4. Conversation messages were maintained using Streamlit session state.
5. Chat conversations were stored using chat_history.json.
6. A New Chat option was implemented for starting separate conversations.
7. Chat-management features such as pinning, renaming, archiving, and deleting conversations were implemented.
8. File and image upload options were incorporated into the application interface.
9. The application provided a user-friendly interface for interacting with a locally hosted LLM.

The implementation therefore covered the main requirements of the assigned Streamlit and local LLM inference task.

10.1.1 Task 1 Observation

The development of Task 1 demonstrated that a locally hosted Large Language Model can be integrated into a web-based application without requiring the chatbot interface itself to directly implement the language-generation process. Streamlit handled the application interface, while Ollama provided access to the locally running model.

The task also demonstrated the importance of conversation state and chat-history management when developing conversational AI applications.

10.2 Task 2 Results

Two medical datasets were experimented with during the fine-tuning process: **MedQuAD** and **MedInstruct-52k**.

The first experiment used `prithvi1029/medquad-medical-qa`, which follows a medical question-answer format. The training loss decreased during the training process, indicating that the model was learning from the provided medical examples.

The second experiment used **MedInstruct-52k**, which follows a medical instruction-response format. In the provided training run, the training loss decreased from approximately **1.3144 at step 10 to 0.9493 at step 250**.

The loss values demonstrate the training behavior of the two experiments, but training loss alone was not considered sufficient to determine the quality of the resulting medical model. The two datasets have different structures and objectives. MedQuAD is oriented toward direct medical question answering, while MedInstruct is designed around medical instruction following.

For the updated implementation, **MedInstruct-52k was used as the final medical fine-tuning dataset**.

10.2.1 Task 2 Observation

Task 2 demonstrated how a pretrained language model can be adapted for a specialized domain without training an entire language model from the beginning.

The workflow also provided practical understanding of how **quantization** can reduce model memory requirements and how **LoRA** can be used as a parameter-efficient approach for model adaptation.

10.3 Combined Results of Month 1

The two tasks provided complementary practical experience.

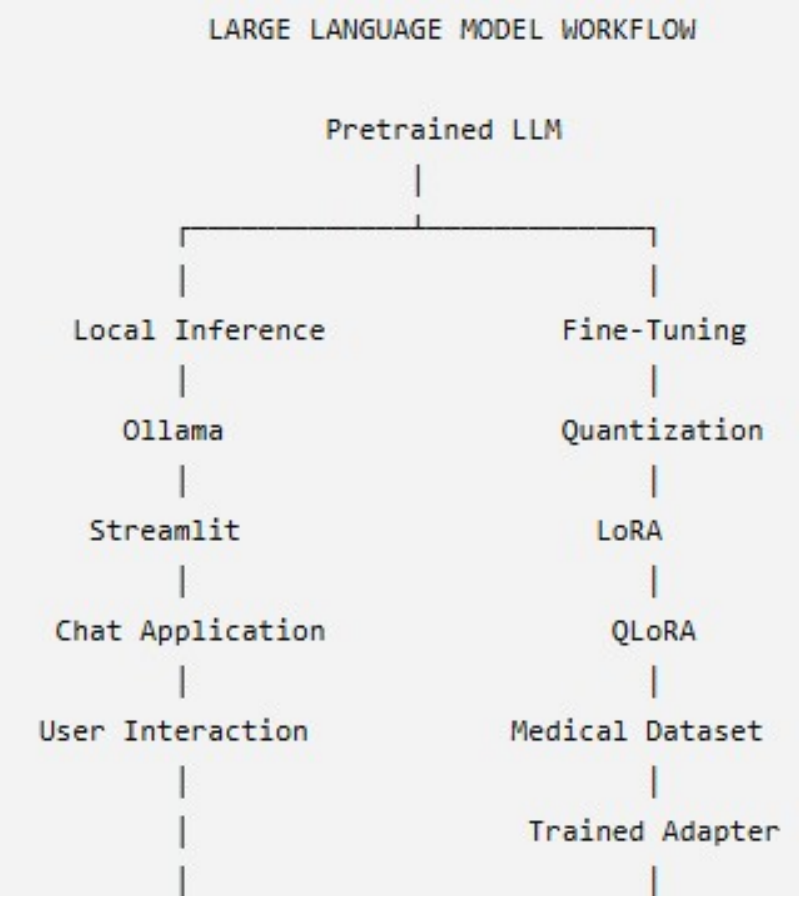
Area	Result
LLM Application	Developed a Streamlit chatbot
Local Model Inference	Implemented through Ollama
Model Used in Task 1	Llama 3.2
Conversation Management	Implemented
Persistent Chat History	Implemented using JSON

Area	Result
Fine-Tuning	Implemented in Google Colab
Fine-Tuning Framework	Unsloth
Base Model for Task 2	Llama 3.2 1B Instruct
Quantization	4-bit
Parameter-Efficient Fine-Tuning	LoRA
Overall Fine-Tuning Approach	QLoRA
Domain	Medical Question Answering
Training Dataset	MedQuAD-based medical QA dataset
Training Examples Used	2,000

10.4 Overall Observation

The Month 1 project provided practical exposure to the complete basic workflow of working with Large Language Models.

The project covered two different stages:



Through Task 1, the focus was on using an LLM in an application**,** while Task 2 focused on adapting an LLM for a specific domain**.**

Together, these tasks helped establish a practical understanding of LLM inference, application development, quantization, parameter-efficient fine-tuning, dataset preparation, and model testing.

Screenshots for Results Section

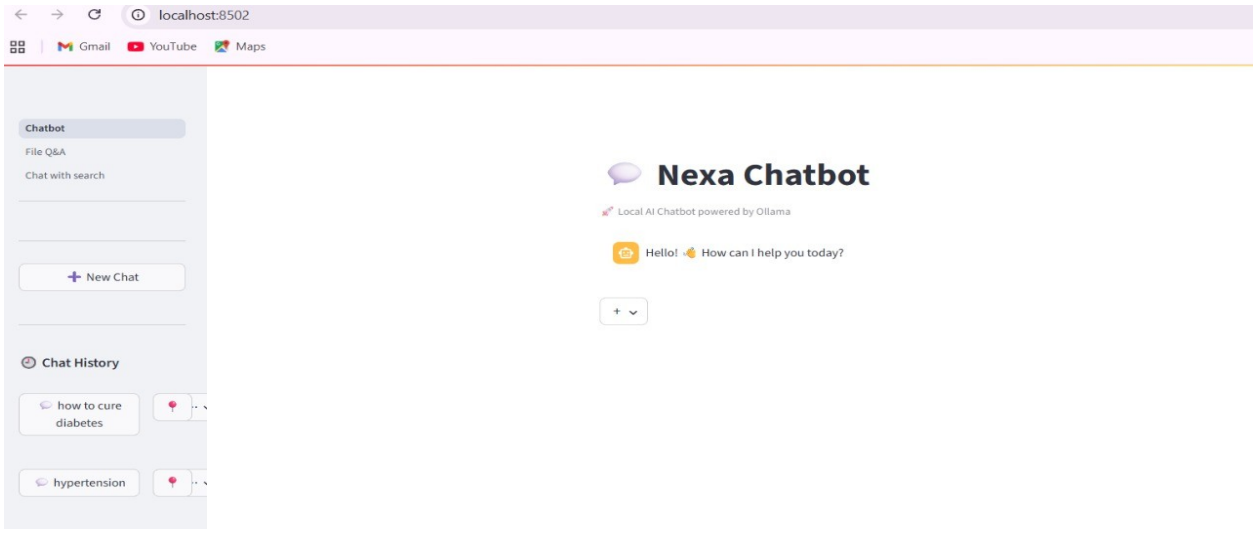


Figure 10.1: Nexa Chatbot home interface

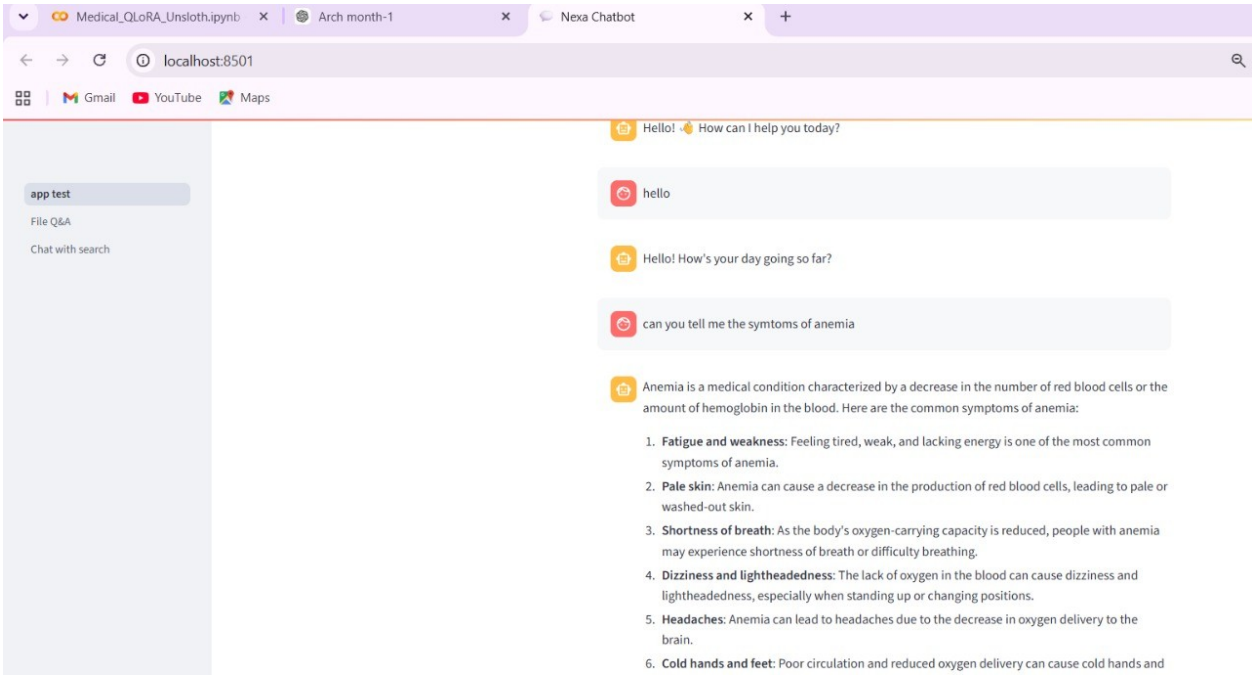


Figure 10.2: User prompt and Llama 3.2 response